SIH26183 — Fund-Trail Investigation Assistant

Full Product Build Plan (Team of 6)

Pitch line: *"From a victim's reported wallet address to a confidence-scored, evidence-backed cash-out report — in under a minute, not a manual multi-hour trace."*

---

1. What We're Keeping vs. Cutting From the Feedback

Feature	Decision	Why
Investigation Copilot / guided case flow	**Keep — build first**	This is the actual product, not a feature
Follow-the-Money animated graph	**Keep — this is the demo centerpiece**	Highest visual impact per hour of build time
Explainable rule-based score (+weights per pattern)	**Keep, but relabel honestly**	Present as a "heuristic scorecard," not "AI explainability" — it's a transparent rule system, which is actually a *selling point*, not a lesser version of ML
Human-in-the-loop confirm/reject on findings	**Keep**	Cheap to build, ethically correct, judges notice this
Evidence hash / audit record	**Keep**	~2 hours of work, disproportionate credibility gain
Red-flag pattern library (fan-out, fan-in, peel chain, splitting, timing burst)	**Keep — these are your actual detection logic**	This is the real technical core, not decoration
Fabricated confidence % breakdowns, invented	**Cut as**	Only ship numbers computed from a real (small)

precision/recall numbers	**shown**	labeled test set — see Section 9
“Investigation Replay” chronological animation	**Stretch goal only**	Nice, not core — build it last, only if MVP is done with time to spare
Multi-chain (15 chains) claims	**Cut**	BTC + ETH only, chain-agnostic architecture stated honestly
Self-scoring tables (9.2 → 10/10)	**Cut entirely**	Not shown to judges, doesn’t inform the build

---

2. MVP Feature Set (must work live in the demo)

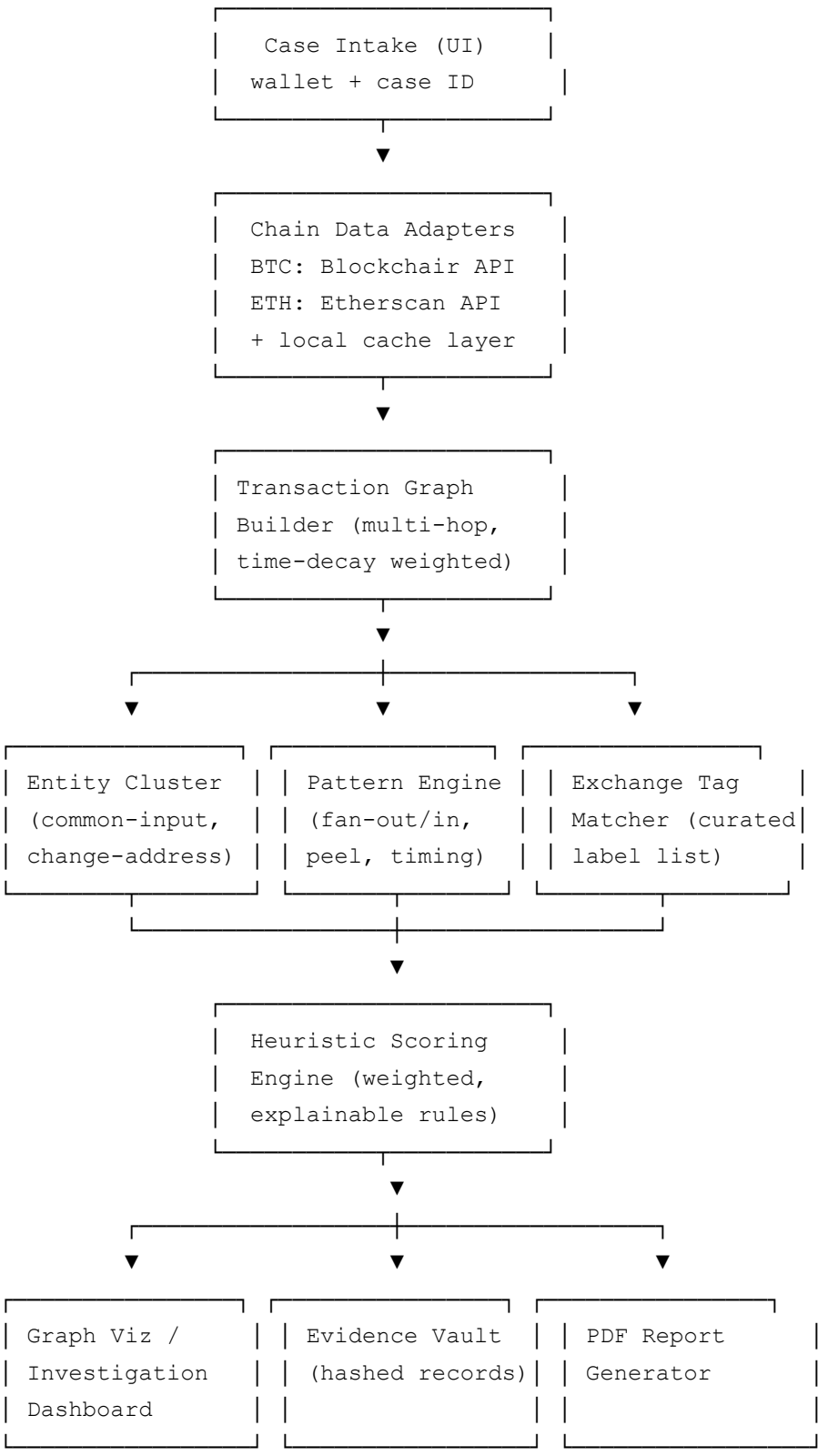
1. Case intake — paste a wallet address + case ID
2. Multi-hop transaction graph pulled from live chain data (real BTC or ETH address)
3. Common-input-ownership + change-address clustering (collapses raw addresses into entity clusters)
4. Heuristic pattern engine flags: fan-out, fan-in, peel chain, round-number splitting, timing burst
5. Rule-based confidence score with a visible “why” breakdown
6. Known-exchange-cluster matching (small curated tag list you build yourself, see Section 6)
7. Follow-the-Money graph visualization, animated by hop
8. One-click PDF report with SHA-256 evidence hash
9. Confirm / Reject / Note on each flagged finding (human-in-the-loop)

Stretch (only if MVP is solid with 6+ hours to spare):

- Chronological replay animation
- Second chain adapter polish (if BTC is MVP, harden ETH or vice versa)
- Case management list view (multiple saved cases)

---

3. System Architecture



Stack suggestion (fast to build in a hackathon window):

- Backend: Python (FastAPI) — graph work is easiest in Python (networkx-style graph

handling)

- Frontend: React + a graph visualization library (e.g., a force-directed graph component) + Tailwind
- DB: PostgreSQL (or SQLite if the team wants zero setup friction for the demo laptop)
- No GPU/ML infra needed for MVP — this is a rules-engine problem, not a deep-learning problem, and that's fine to say out loud to judges

---

4. Frontend Screens

Screen	Purpose
Case Intake	Enter wallet address, chain, case ID. Big "Investigate" button.
Investigation Dashboard	Summary card: wallets discovered, transactions analyzed, clusters found, overall risk score
Follow-the-Money Graph View	The centerpiece — animated, hop-by-hop fund flow, cluster nodes collapse visually
Why? Panel	Click any score → see the pattern breakdown (fan-out +X, peel chain +Y, etc.) with the actual transaction hashes behind each flag
Findings Review	List of flagged patterns with Confirm / Reject / Add Note controls
Report View / Export	Final one-page report preview + "Generate PDF" + evidence hash shown

---

5. Backend API Endpoints (indicative)

```
POST   /cases                          create a new case
POST   /cases/{id}/investigate          trigger the pipeline for a wallet address
GET    /cases/{id}/graph                fetch the built transaction graph
GET    /cases/{id}/clusters             fetch entity clusters
GET    /cases/{id}/findings             fetch flagged patterns + scores
POST   /cases/{id}/findings/{fid}/review   confirm/reject/note a finding
GET    /cases/{id}/report               generate/fetch the PDF report
GET    /cases/{id}/evidence/{eid}/verify   verify a piece of evidence against its
```

6. Database Schema (core tables)

```
cases(id, complaint_id, wallet_address, chain, created_at, status)

wallets(id, case_id, address, first_seen, last_seen, cluster_id)

transactions(id, tx_hash, from_wallet_id, to_wallet_id, amount,
             timestamp, chain, hop_depth)

clusters(id, case_id, cluster_label, member_wallet_ids, exchange_tag_match)

findings(id, case_id, pattern_type, score_contribution,
         evidence_tx_ids, status[flagged/confirmed/rejected], analyst_note)

exchange_tags(address, exchange_name, source, confidence)

evidence_records(id, case_id, payload_hash, created_at, verified)

reports(id, case_id, pdf_path, generated_at, evidence_hash)
```

7. Blockchain Data Pipeline — Real API Details

Bitcoin: Blockchair API

- No key needed for light testing, but rate-limited to **30 requests/minute** on the free plan (soft cap of 5 req/sec applied only under heavy server load).
- Apply for a free non-commercial/academic API key ahead of time (email-based approval) — don't leave this to the night before.
- Cache every response locally (SQLite/Redis) — you will re-query the same addresses multiple times during graph expansion, and burning your rate limit mid-demo is the #1 avoidable failure mode.

Ethereum: Etherscan API (V2)

- Free tier: **5 calls/second**, daily call cap (historically ~100k/day, though Etherscan has been tightening free-tier limits through 2026 — check current terms before the event and get a key early).

- Historical/name-tag endpoints are capped separately at ~2 calls/sec regardless of tier.
- Max records per request has been reduced on the free tier recently — paginate rather than assuming one large pull works.

Practical rule for the hackathon: build a caching layer from hour one. Pre-fetch and cache the transaction graph for your demo wallet(s) well before you're on stage — never depend on a live external API call during the actual judge-facing demo. Live-fetch for credibility during Q&A, cached for the scripted demo.

---

8. Pattern Engine — The Actual Detection Logic

Implement these as explicit, inspectable rules (not a black box):

Pattern	Detection logic (simplified)	Suggested weight
Rapid fan-out	One address sends to N≥5 new addresses within a short time window	+20
Fan-in / consolidation	N≥5 addresses send to one address within a short window	+15
Peel chain	A chain of transactions where each hop sends a small amount forward and returns the remainder to a new "change" address	+20
Round-number splitting	An amount is split into near-equal, round-number pieces	+15
Timing burst	Multiple transactions from related addresses within seconds/minutes of each other	+12
Known exchange cluster match	Destination cluster matches a curated tag list	+18 (redirects to attribution, not just risk)

Be explicit in your pitch: these weights are a starting heuristic, tunable, and open to being replaced by a trained model later — don't claim they came from a dataset unless they actually did.

---

9. Evaluation — How to Get Real Numbers Without Inventing Them

1. Build a small labeled test set (15–30 cases) from **publicly documented** cases: known past exchange hacks, mixer cases, or scam wallets where researchers or news reports already published the laundering path (e.g., widely covered hack post-mortems). This gives you real ground truth you're allowed to use.
2. Run your pipeline against these and manually check: did it correctly flag the pattern, correctly attribute the cluster, or miss it?
3. Report only what you actually measured — even something modest like "flagged 11/14 known laundering patterns correctly in our test set, 2 false positives" is far more credible than a polished-looking but fake 92%/91% table.
4. Time comparison is safe and easy to measure honestly: run a manual trace yourself on one case, time it, compare to your system's time. This is a legitimate, unfakeable number.

---

10. Team Split (6 members, by function — map to your actual people)

Role	Focus
Backend / Graph Engine (1–2 people)	Chain adapters, graph builder, clustering logic
Pattern & Scoring Engine (1 person)	The heuristic rules, weight tuning, evaluation set
Frontend / Visualization (1–2 people)	Dashboard, Follow-the-Money graph, Why panel
Data & Attribution (1 person)	Building the curated exchange-tag list, caching layer, API key management
Report/Evidence + Pitch Lead (1 person)	PDF generation, evidence hashing, demo script, slide deck, judge Q&A prep

(Adjust based on who's actually strong where — this is a functional split, not a fixed assignment.)

11. Suggested Build Timeline

Before the hackathon (prep days):

- Get Blockchair + Etherscan API keys approved
- Pick and pre-validate 2–3 real demo wallet addresses (one clean, one with a real historical mixing/scam pattern if you can find one publicly documented)
- Build the caching layer and confirm it works offline once cached
- Build your labeled evaluation set (Section 9) ahead of time — don't leave this for hour 30

Hours 0–8: Chain adapters + graph builder + basic clustering, wired end-to-end (even if ugly) **Hours 8–16:** Pattern engine + scoring, connected to the graph **Hours 16–24:** Frontend dashboard + Follow-the-Money visualization **Hours 24–30:** Report generation, evidence hashing, Confirm/Reject UI **Hours 30–34:** Run the evaluation set, get real numbers, polish the demo script **Hours 34–36:** Rehearse the pitch end-to-end, buffer for bug fixes

---

12. 5-Minute Demo Script (grounded, not hyped)

0:00–0:30 — "A citizen reports a crypto scam through India's cybercrime helpline. All the investigator has is one wallet address." **0:30–1:00** — Paste the address, hit Investigate. Real API call (or cached, stated as such) runs live. **1:00–1:45** — Graph builds on screen: "X wallets discovered, Y transactions analyzed, Z clusters formed" — real numbers from this run, not fixed to a script. **1:45–2:30** — Pattern flags appear: fan-out, peel chain, etc. Click one → show the Why panel with the actual transaction hashes. **2:30–3:00** — Cluster collapses to the most likely exit cluster, matched against your curated exchange tag list, confidence score shown. **3:00–3:30** — Click Generate Report → PDF appears with the evidence hash. **3:30–4:15** — Show the evaluation numbers you actually measured (Section 9) — this is where you earn technical credibility. **4:15–5:00** — State limitations honestly: cross-chain/mixer-hop tracing degrades confidence (say so, don't hide it), attribution data is a curated subset not exhaustive, this is an investigator-assist tool with human review built in, not an automated accusation engine.

---

13. What to Say Upfront to Judges (builds trust fast)

- "This is decision support for an investigator, not an automated verdict — every flag

requires human confirmation."

- "Our confidence scores come from an explicit, inspectable rule set — we can show you exactly why any score was given."
- "We validated this against publicly documented past cases; here are the real numbers, not estimates."
- "The architecture is chain-agnostic; we built and validated Bitcoin and Ethereum for this prototype."